

Informe de Arquitectura y Decisiones del Notebook si7011-dl-mlp-pytorch

Modelo de Clasificación de Imágenes con MLP

El notebook implementa un modelo de clasificación de imágenes utilizando redes neuronales completamente conectadas (MLP) en PyTorch.

El objetivo principal no es alcanzar el mejor desempeño posible, sino **entender los fundamentos del deep learning**, especialmente el flujo completo de datos y el comportamiento de una red neuronal básica aplicada a imágenes.

Transformaciones aplicadas

Las imágenes pasan por el siguiente pipeline:

- Redimensionamiento a 150x150
- Conversión a escala de grises
- Aumentos de datos (flip y rotación)
- Normalización

Se crean los dataloaders de entrenamiento, test y validación. Se les aplica shuffle para generalizar el proceso de entrenamiento, igualmente un copy a la gpu sin bloqueo para permitir el copiado de los batches desde la cpu sin necesidad de frenar el proceso de entrenamiento.

Representación de la Imagen (Flattening)

Antes de entrar al modelo, las imágenes son transformadas a un vector: `nn.Flatten()`
Esto convierte una imagen de:

$1 \times 150 \times 150 \rightarrow 22500$ características

¿Por qué se hace esto?

- Permite usar un MLP en lugar de una CNN

Arquitectura del Modelo

Se implementa una red neuronal tipo MLP con múltiples capas:

- Entrada: 22,500 features
- Capas ocultas:
 - 1024 neuronas
 - 512 neuronas
 - 256 neuronas
- Salida: 6 clases

Componentes clave:

- **ReLU**: introduce no linealidad
- **BatchNorm**: estabiliza el entrenamiento
- **Dropout**: reduce overfitting

Función de Pérdida y Optimización

- **Loss**: CrossEntropyLoss
 - **Optimizador**: SGD con momentum para evitar que el gradiente quede estático o bluea
- ```
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

### Flujo:

1. Forward pass
2. Cálculo de pérdida
3. Backpropagation
4. Actualización de pesos

## Limitaciones del Enfoque

- No aprovecha la estructura espacial (como lo haría una CNN)
- Puede tener menor accuracy comparado con modelos modernos
- No usa arquitecturas especializadas en visión

## Próximos Pasos

Para mejorar el modelo en futuras iteraciones:

- Usar redes convolucionales (CNN)
- Mantener imágenes en RGB
- Aplicar transfer learning (ej: ResNet, EfficientNet)
- Ajustar hiperparámetros